

Memory Efficient Versioned File Indexer Using C++

NAME : KATROTH CHANDRAKANTH

ROLL NO : 240531

Overview of the Problem

In this assignment, the goal was to build a memory efficient system that can process large text files and create a word level index. The main challenge is that the program should not load the entire file into memory. Instead, the file must be processed gradually using a fixed size buffer.

The program reads a text file, extracts words from it, and keeps track of how many times each word appears. The system also supports multiple versions of files so that we can compare results between different files.

After building the index, the program supports three types of queries:
word frequency query, top k frequent words query, and difference query between two versions.

Design and Approach

To solve the problem, I designed the program using multiple classes so that each class handles a specific task.

The main idea is to read the file in small chunks using a fixed size buffer. Each chunk is then processed by a tokenizer that extracts valid words from the text. The words are converted to lowercase so that matching is case insensitive.

Every extracted word is stored in a versioned index where the key is the word and the value is the number of times the word appears. Each dataset is stored as a different version so that queries can be run on multiple files.

Queries are implemented using an abstract base class and derived query classes. This allows the program to choose the correct query type at runtime

Description of the Classes Used

The program uses several classes to keep the design organized.

bufferread : This class is responsible for reading the file using a fixed size buffer. Instead of loading the entire file, it reads the file chunk by chunk.

Tokenizer : This class processes each chunk of text and extracts valid words. Words are defined as sequences of alphanumeric characters. All words are converted to lowercase before being stored.

versionedindex : This class stores the word frequency index for each version. Internally it uses hash maps to map each word to its count.

query : This is an abstract base class that defines the structure for executing queries.

wordquery : This class returns the number of times a specific word appears in a version.

diffquery : This class compares two versions of the dataset and returns the difference in frequency for a given word.

topquery : This class returns the top k most frequent words from a version.

C++ Requirements and Base Class

The program also follows the C++ requirements mentioned in the assignment. I used multiple classes with clear roles such as `bufferread` for reading the file with a buffer, `Tokenizer` for extracting words, and `versionedindex` for storing the word counts.

For inheritance and runtime polymorphism, I created an abstract base class called `query`. The classes `wordquery`, `diffquery`, and `topquery` inherit from this class and implement the `execute` function to run different queries. I also used a template function `printm` so it can work with different data structures like maps and vectors. Function overloading is shown by using two versions of `printm`. Exception handling is used in places like file opening and buffer validation to handle possible runtime errors.

How the File is Processed Using the Fixed Size Buffer

The file is processed using a fixed size buffer between 256 KB and 1024 KB.

The buffer reader reads a portion of the file and stores it in memory. This chunk is then passed to the tokenizer which extracts words from the text.

Each word is then added to the index and its frequency is updated. This process continues until the entire file has been processed.

Using this method ensures that the program does not depend on the size of the file. Even if the dataset is very large, only a small portion of it is loaded into memory at a time.

Assumptions and Observations

Some assumptions were made while implementing the system.

Words are defined as contiguous sequences of alphanumeric characters.

Word matching is case insensitive so uppercase and lowercase words are treated the same.

The dataset can be very large so the program processes the file incrementally using a buffer.

Memory usage grows only with the number of unique words and not with the size of the file.

Command line arguments are used to specify the file path, version name, buffer size, and query type.

Conclusion

Overall, the program successfully builds a memory efficient word index for large text files. By using buffered reading and incremental processing, the system avoids loading the entire file into memory.

The object oriented design also makes the code easier to organize and extend. The system supports multiple query types and can handle multiple file versions within the same execution.